Filament Get, Set, Repeater, and Table State Notes

Project-specific notes for the Kaido Kit order form in `app/Filament/Resources/OrderResource.php`. These notes explain how Filament reads form values, writes computed values, calculates totals inside a repeater, and displays calculated table columns.

Laravel | Filament Forms | Get | Set | Repeater | Table State

Main file
`app/Filament/Resources/OrderResource.php`

Related database table
`order_items`

Main form relationship
`Order hasMany OrderItem`

Main goal
Automatically compute order totals from selected products and quantities.

---

1. The Big Idea

Filament forms keep a temporary copy of the form data while the user is typing. This temporary data is called the form state. `Get` reads from that state. `Set` writes to that state.

```
use Filament\Forms\Get;
use Filament\Forms\Set;
```

Helper	Purpose	Simple meaning
`Get` `$get`	Reads the current value of another field.	`$get('product_id')` means "what product is selected right now?"
`Set` `$set`	Changes the current value of another field.	`$set('purchase_price', 150)` means "put 150 into the purchase price field."

> **Important:** `Get` and `Set` work on the form state first. They do not immediately save to the database. The values are saved when the form is submitted and Filament dehydrates the form state.

2. Project Data Structure

Your order form is built around these models and relationships:

Model	Important fields	Relationship
`Order`	`user_id`, `total`, `amount_tendered`	`Order` has many `OrderItem` records through `orderItems()`
`OrderItem`	`order_id`, `product_id`, `purchase_price`, `quantity`	`OrderItem` belongs to `Order` and belongs to `Product`
`Product`	`name`, `description`, `cost_price`	Selected by `product_id` inside each order item row

The `order_items` migration confirms that `purchase_price` and `quantity` are numeric decimal columns:

```
$table->foreignId('order_id')->constrained();
$table->foreignId('product_id')->constrained();
$table->decimal('purchase_price', 10, 2);
$table->decimal('quantity', 10, 2);
```

3. What The Form State Looks Like

When a user creates an order, Filament may hold form data like this before saving:

```
[
    'user_id' => 1,
    'orderItems' => [
        [
            'product_id' => 1,
            'purchase_price' => 15.00,
            'quantity' => 2,
        ],
        [
            'product_id' => 2,
            'purchase_price' => 20.00,
            'quantity' => 3,
        ],
    ],
    'total' => 90.00,
    'amount_tendered' => 100.00,
    'change' => 10.00,
]
```

That array is the idea students should imagine when they see `$get()` and `$set()`.

---

4. The Repeater In This Project

Your order items are inside this repeater:

```
Repeater::make('orderItems')
    ->relationship()
    ->schema([
        Select::make('product_id'),
        TextInput::make('purchase_price'),
        TextInput::make('quantity'),
    ])
```

The name `orderItems` is not random. It matches the relationship method in the `Order` model:

```
public function orderItems(): HasMany
{
    return $this->hasMany(OrderItem::class);
}
```

Because of `->relationship()`, Filament knows that every row inside the repeater should be saved as an `OrderItem` record connected to the current `Order`.

5. Relative Paths In A Repeater

The most confusing part for students is usually this line:

```
$items = $get('../../orderItems');
```

Inside a repeater row, `$get()` starts from the current row. So if the callback is inside `quantity`, then:

Code	Meaning
`$get('product_id')`	Get the selected product in the current repeater row.
`$get('quantity')`	Get the quantity in the current repeater row.
`$get('../')`	Move up from the current field toward the current row state.
`$get('../../orderItems')`	Move up to the parent form area, then read the whole `orderItems` repeater.

> **Student checkpoint:** If you only need data from the same row, use names like `product_id` or `quantity`. If you need all rows, use a parent path such as `../../orderItems`.

6. The Calculation Callback

Your code defines one reusable function for calculating the order total:

```
$calculations = function (Get $get, Set $set) {
    $items = $get('../../orderItems');
    $total = 0;

    if (is_array($items)) {
        foreach ($items as $item) {
            $product = Product::find($item['product_id'] ?? null);

            if ($product) {
                $total += $product->cost_price * (float) ($item['quantity'] ?? 0);
            }
        }
    }

    $set('total', $total);
};
```

Line-by-line explanation

Code	Explanation
`function (Get $get, Set $set)`	Filament injects the `Get` and `Set` helpers into the callback.
`$items = $get('../../orderItems');`	Read all repeater rows from the form state.
`$total = 0;`	Start the total at zero before adding each item subtotal.
`if (is_array($items))`	Make sure the repeater data is an array before looping.
`foreach ($items as $item)`	Loop through every order item row.
`Product::find($item['product_id'] ?? null)`	Find the selected product from the database. If no product is selected, use `null`.
`$product->cost_price * quantity`	Multiply product price by the quantity entered by the user.
`$set('total', $total)`	Write the calculated total back into the form state.

Example calculation

Suppose the user adds two order items:

Product	Price	Quantity	Subtotal
Product 1	15.00	2	30.00
Product 2	20.00	3	60.00

```
Total = 30.00 + 60.00
Total = 90.00
```

---

7. Project-Specific Correction For Total Path

The `total` field is outside the repeater:

```
TextInput::make('total')
    ->numeric()
    ->disabled()
    ->dehydrated()
```

Because the calculation callback runs from inside the repeater, it is safer to set the parent form's `total` field using:

```
$set('../../total', $total);
```

Instead of:

```
$set('total', $total);
```

> **Why this matters:** From inside a repeater row, `$set('total', $total)` can mean "set a field named total inside this current row." But your actual `total` field belongs to the main order form, outside `orderItems`.

8. Teaching-Friendly Calculation Version

For students, this version is easier to understand because it uses the row's saved `purchase_price` and `quantity` directly:

```
$calculations = function (Get $get, Set $set) {
    $items = $get('../../orderItems') ?? [];
    $total = 0;

    foreach ($items as $item) {
        $price = (float) ($item['purchase_price'] ?? 0);
        $quantity = (float) ($item['quantity'] ?? 0);

        $total += $price * $quantity;
    }

    $set('../../total', $total);
};
```

This also matches the table calculation later:

```
$item->purchase_price * $item->quantity
```

9. Product Select: Using `afterStateUpdated`

Your product field is:

```
Select::make('product_id')
    ->relationship('product', 'name')
    ->live(debounce: 1000)
    ->afterStateUpdated(function (Set $set, $state) {
        $product = Product::find($state);
        $set('purchase_price', $product->product_name ?? 0);
    }, $calculations)
```

Here is what each part means:

Code	Meaning
`Select::make('product_id')`	This field stores the selected product's ID in `order_items.product_id`.
`->relationship('product', 'name')`	Use the `product()` relationship from `OrderItem`, and display the product's `name`.
`->live(debounce: 1000)`	Send the updated value to Filament after the user changes it. The 1000 means wait around one second.
`$state`	The new selected product ID.
`Product::find($state)`	Find the chosen product record.
`$set('purchase_price', ...)`	Put the product price into the same repeater row's `purchase_price` field.

Important project-specific fix

Your `products` table uses `name` and `cost_price`. It does not have `product_name`. Also, your `purchase_price` column is decimal, so it should receive a price, not a product name.

```
$set('purchase_price', $product?->cost_price ?? 0);
```

Recommended version

In the installed Filament code in this project, `afterStateUpdated()` accepts one callback. To run both "set purchase price" and "recalculate total", place both actions inside one callback:

```
Select::make('product_id')
    ->relationship('product', 'name')
    ->live(debounce: 1000)
    ->afterStateUpdated(function (Get $get, Set $set, $state) use ($calculations) {
        $product = Product::find($state);

        $set('purchase_price', $product?->cost_price ?? 0);

        $calculations($get, $set);
    })
    ->required()
    ->searchable();
```

10. Quantity Field: Disabling Until Product Is Selected

Your quantity field contains this condition:

```
->disabled(fn (Get $get) => $get('product_id') === null)
```

This means:

```
If product_id is null:
    disable quantity

If product_id has a value:
    enable quantity
```

Because this code is inside a repeater row, `$get('product_id')` reads the product selected in the same row.

> **Student checkpoint:** The quantity input depends on the product select. This prevents the user from typing quantity before choosing a product.

11. Quantity Field: Recalculating After Typing

Your quantity field also has:

```
->live(debounce: 1000)
->afterStateUpdated($calculations)
```

When the user changes quantity, Filament waits about one second, sends the new value to the server, and then runs the calculation callback.

The flow is:

1. User selects a product.
2. Filament sets `purchase_price`.
3. User enters quantity.
4. `afterStateUpdated($calculations)` runs.
5. The callback reads all repeater rows.
6. The callback computes the total.
7. `Set` writes the result into the `total` field.

12. Disabled Fields And `dehydrated()`

Your `purchase_price` field is:

```
TextInput::make('purchase_price')
    ->disabled()
    ->dehydrated()
    ->numeric()
```

Your `total` field is:

```
TextInput::make('total')
    ->numeric()
    ->disabled()
    ->dehydrated()
```

This is important because disabled fields are normally not editable by the user. Adding `->dehydrated()` tells Filament to still include the value when saving the form.

Field	Why disabled?	Why dehydrated?
`purchase_price`	The system should fill it from the selected product.	It must be saved to `order_items.purchase_price`.
`total`	The system should calculate it automatically.	It must be saved to `orders.total`.

13. Amount Tendered Validation

Your `amount_tendered` field is disabled until a valid total exists:

```
->disabled(fn (Get $get) =>
    $get('total') === null || (float) $get('total') === 0.0
)
```

This means the cashier cannot enter payment until the order has a total.

The custom rule checks that the payment is enough:

```
->rules([
    fn (Get $get): Closure => function (string $attribute, $value, Closure $fail) use ($g
        $total = (float) $get('total');
        $amountTendered = (float) $value;

        if ($total > $amountTendered) {
            $fail('Amount tendered should be greater than or equal to total.');
        }
    },
])
```

The logic is:

```
If total is greater than amount tendered:
    show validation error
```

---

14. Computing Change With `Set`

Your amount tendered field calculates change after it is updated:

```
->afterStateUpdated(function (Set $set, Get $get, $state) {
    $change = (float) $get('amount_tendered') - (float) $get('total');
    $set('change', $change);
})
```

Here, `Get` reads the payment and total. Then `Set` writes the computed change into the `change` field.

Example:

```
amount_tendered = 100
total = 90

change = 100 - 90
change = 10
```

In your current database migration, `change` is not part of the `orders` table. So this field can be shown on the form without being saved, unless you later add a `change` column.

15. Form Callbacks Vs Table State

`Get` and `Set` are for form state. The table uses a different idea: `state()`.

In your table columns:

```
TextColumn::make('sub_total')
    ->state(fn ($record) => $record->orderItems->map(
        fn ($item) => $item->purchase_price * $item->quantity
    ))
    ->listWithLineBreaks()
    ->money('PHP')
    ->label('Sub Total')
```

Here, `$record` is not form state. It is an actual `Order` model loaded from the database.

Form	Table
Uses `Get` and `Set`.	Uses `$record` and `state()`.
Works before saving, while the user is editing.	Works after saving, while displaying database records.
Reads temporary form values.	Reads model values from the database.

16. `map()` For Subtotals

This code:

```
->state(fn ($record) => $record->orderItems->map(
    fn ($item) => $item->purchase_price * $item->quantity
))
```

means:

```
For every order item:
    compute purchase_price * quantity
    return each computed value
```

Example:

```
$record->orderItems = [
    ['purchase_price' => 15, 'quantity' => 2],
    ['purchase_price' => 20, 'quantity' => 3],
];
```

The `map()` result is:

```
[
    30,
    60,
]
```

Because you added `->listWithLineBreaks()`, Filament displays those values on separate lines.

---

17. `sum()` For Total

This code:

```
->state(fn ($record) => $record->orderItems->sum(
    fn ($item) => $item->purchase_price * $item->quantity
))
```

means:

```
For every order item:
    compute purchase_price * quantity
    add all results together
```

Using the same example:

```
15 * 2 = 30
20 * 3 = 60

sum = 30 + 60
sum = 90
```

> **Teaching point:** `map()` gives many results. `sum()` gives one final number. If you wanted the highest subtotal only, that would be `max()`, not `sum()`.

18. Why Store `purchase_price` In Order Items?

The `order_items` table stores `purchase_price` even though the product already has `cost_price`. This is a common order system pattern.

Reason: product prices can change in the future. If a product costs 15 today and becomes 20 next month, old orders should still show the price at the time of purchase.

Field	Meaning
`products.cost_price`	The current product price.
`order_items.purchase_price`	The price copied into the order at checkout time.

19. Full Recommended Teaching Snippet

This snippet keeps the code aligned with your current project structure:

```
$calculations = function (Get $get, Set $set) {
    $items = $get('../../orderItems') ?? [];
    $total = 0;

    foreach ($items as $item) {
        $price = (float) ($item['purchase_price'] ?? 0);
        $quantity = (float) ($item['quantity'] ?? 0);

        $total += $price * $quantity;
    }

    $set('../../total', $total);
};
```

```
Select::make('product_id')
    ->relationship('product', 'name')
    ->live(debounce: 1000)
    ->afterStateUpdated(function (Get $get, Set $set, $state) use ($calculations) {
        $product = Product::find($state);

        $set('purchase_price', $product?->cost_price ?? 0);

        $calculations($get, $set);
    })
    ->required()
    ->searchable()
    ->columnSpanFull();
```

```
TextInput::make('quantity')
    ->numeric()
    ->live(debounce: 1000)
    ->disabled(fn (Get $get) => $get('product_id') === null)
    ->afterStateUpdated($calculations)
    ->required();
```

20. Common Student Mistakes

Mistake	Why it happens	Fix
Using `$get('orderItems')` inside a repeater row	The callback starts inside the current row, so it cannot find the parent repeater directly.	Use `$get('../../orderItems')`.
Using `$set('total', $total)` from inside a repeater	It may target the current row instead of the parent form.	Use `$set('../../total', $total)`.
Setting `purchase_price` to product name	The field name may confuse beginners.	Set it to `$product?->cost_price ?? 0`.
Forgetting `->live()`	The callback may not run when the student expects.	Add `->live(debounce: 1000)` to fields that should react.
Forgetting `->dehydrated()` on disabled calculated fields	Disabled fields are display-only by default behavior.	Add `->dehydrated()` when the value must be saved.
Thinking table `state()` uses `Get` and `Set`	Both are named "state", but they happen in different places.	Remember: forms use `Get` / `Set`; tables use `$record`.

21. Vocabulary For Students

Term	Simple definition
State	The current value of a form field or component.
Get	Read a form value.
Set	Write a form value.
Repeater	A form component that lets the user add multiple rows of related data.
Relationship repeater	A repeater that saves rows to a related model, like `OrderItem`.
Live field	A field that sends updates to Filament while the user is editing.
Debounce	A delay before sending the update, useful to avoid too many requests.
Dehydrated	Included when Filament prepares the data for saving.
Table state	A displayed value for a table column, often calculated from the model record.

22. Mini Exercises

1. Explain why `$get('product_id')` works inside the quantity field.
2. Explain why `$get('../../orderItems')` is needed to calculate the whole order total.
3. Change the calculation to use `purchase_price` instead of querying `Product::find()` inside the loop.
4. Given `purchase_price = 12` and `quantity = 5`, compute the item subtotal.
5. Explain the difference between `map()` and `sum()` in the table columns.
6. Explain why `purchase_price` should be saved even if the product price changes later.

23. Short Quiz With Answers

Question	Answer
What does `$get('product_id')` do?	It reads the current selected product ID from the form state.
What does `$set('purchase_price', 50)` do?	It places `50` into the `purchase_price` field.
What is `$state` inside the product select callback?	The newly selected product ID.
Why does the total calculation loop through `orderItems`?	Because the order can have multiple products, and each row contributes to the total.
What does `map()` return?	A collection of transformed values, such as one subtotal per order item.
What does `sum()` return?	One number, the total of all computed values.

24. Final Mental Model

Teach students to remember this simple flow:

```
User changes product or quantity
        |
        v
Filament updates form state because the field is live
        |
        v
afterStateUpdated() runs
        |
        v
Get reads the needed values
        |
        v
PHP calculates the result
        |
        v
Set writes the result back into another field
        |
        v
Filament saves dehydrated fields when the form is submitted
```

The table display is separate:

```
Order is already saved
        |
        v
Filament table loads $record
        |
        v
state() calculates display values from $record->orderItems
        |
        v
map() shows each subtotal, sum() shows one total
```

Prepared for the Kaido Kit Laravel project. The examples are based on the current order form, order item migration, and model relationships in this project folder.